

DBMS LAB EXPERIMENTS

EXPERIMENT 1:

AIM: Creation, altering and dropping of tables and inserting rows into a table using constraints and SELECT command.

CONSTRAINTS

Constraints enforce limits on the data or type of data that can be **inserted, updated, or deleted** from a table.

The main purpose of constraints is to **maintain data integrity** in a table.

Types of Constraints

NOT NULL

UNIQUE

DEFAULT

CHECK

Key Constraints

PRIMARY KEY

FOREIGN KEY

NOT NULL Constraint

NOT NULL constraint ensures that a column **cannot store NULL values**.

Syntax:

```
CREATE TABLE Student (  
    student_id INT,  
    name VARCHAR(50) NOT NULL  
);
```

UNIQUE Constraint

UNIQUE constraint ensures that all values in a column are different.

Syntax:

```
CREATE TABLE Student (  
    email VARCHAR(100) UNIQUE  
);
```

DEFAULT Constraint

DEFAULT constraint assigns a default value to a column when no value is provided.

Syntax:

```
CREATE TABLE Student (  
    city VARCHAR(30) DEFAULT 'Hyderabad'  
);
```

CHECK Constraint

CHECK constraint ensures that values in a column satisfy a specific condition.

Syntax:

```
CREATE TABLE Student (  
    age INT CHECK (age >= 18)  
);
```

PRIMARY KEY Constraint

PRIMARY KEY constraint uniquely identifies each record in a table and does not allow NULL values.

Syntax:

```
CREATE TABLE Student (  
    Rollno INT PRIMARY KEY,  
    name VARCHAR(50)  
);
```

FOREIGN KEY Constraint

FOREIGN KEY constraint is used to link two tables and ensures that values in a column exist in the parent table.

Syntax:

Parent Table:

```
CREATE TABLE Dept (  
    deptcode INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

Child table:

```
CREATE TABLE Student (  
    Rollno INT PRIMARY KEY,  
    name VARCHAR(50),  
    deptcode INT,  
    FOREIGN KEY (deptcode) REFERENCES Dept(deptcode)  
);
```

CREATE TABLE (with Constraints)**Syntax:**

```
CREATE TABLE table_name (  
    column_name datatype [constraint],  
    column_name datatype [constraint]  
);
```

```
SQL> CREATE TABLE student (  
    sid INT PRIMARY KEY,  
    name VARCHAR2(20) NOT NULL,  
    marks INT CHECK (marks BETWEEN 0 AND 100),  
    email VARCHAR2(30) UNIQUE  
);
```

Output:?**Check table structure:****Syntax:**

```
DESCRIBE table_name;
```

```
SQL> DESCRIBE student;
```

Output: ?

INSERT VALUES INTO TABLE**Syntax:**

```
INSERT INTO table_name VALUES (value1, value2, ...);
```

```
SQL> INSERT INTO student VALUES (1, 'Ravi', 85, 'ravi@gmail.com');  
SQL> INSERT INTO student VALUES (2, 'Sita', 92, 'sita@gmail.com');  
SQL> INSERT INTO student VALUES (3, 'Amit', 78, 'amit@gmail.com');
```

```
SQL> INSERT INTO student VALUES (4, 'Neha', 88, 'neha@gmail.com');
SQL> INSERT INTO student VALUES (5, 'Kiran', 95, 'kiran@gmail.com');
```

Output:?

DISPLAY TABLE DATA (SELECT)

Syntax:

```
SELECT * FROM table_name;
```

```
SQL> SELECT * FROM student;
```

Output:?

ALTER Command:

ALTER is a DDL (Data Definition Language) command used to modify the structure of an existing table without losing data. It is mainly used to add, modify, or delete columns and constraints of a table.

ALTER TABLE (ADD COLUMN)

Syntax:

```
ALTER TABLE table_name ADD column_name datatype;
```

Query: Write a query to add a new column 'dept' to the student table.

```
SQL> ALTER TABLE student ADD dept VARCHAR2(20);
```

Output:?

Check table structure:

```
SQL> DESCRIBE student;
```

Output:?

ALTER TABLE (MODIFY COLUMN)

Syntax:

```
ALTER TABLE table_name MODIFY column_name datatype;
```

Query: Write a query to modify the name column of the student table.

```
SQL> ALTER TABLE student MODIFY name VARCHAR2(30);
```

Output: ?

ALTER TABLE (DROP COLUMN)

Syntax:

ALTER TABLE table_name DROP COLUMN column_name;

Query: Write a query to drop the dept column from the student table.

SQL> ALTER TABLE student DROP COLUMN dept;

Output:?

ALTER TABLE (RENAME COLUMN)

Syntax:

ALTER TABLE table_name RENAME COLUMN old_name TO new_name;

Query: Write a query to rename the marks column to score.

SQL> ALTER TABLE student RENAME COLUMN marks TO score;

Output:?

ALTER TABLE (ADD CONSTRAINT)

Constraint names are required to uniquely identify constraints in a table. If a constraint name is not provided, Oracle generates a system-defined name, which is difficult to remember and manage. Providing a constraint name makes it easy to modify or drop constraints later.

Query: Write a query to add constraints to the student table.

Method 1: ADD constraint WITHOUT constraint name

Syntax:

ALTER TABLE table_name ADD PRIMARY KEY (column_name);

Query: Write a query to add a primary key to the student table

SQL> ALTER TABLE student ADD PRIMARY KEY (sid);

Output?

Syntax:

ALTER TABLE table_name ADD UNIQUE (column_name);

Query: Write a query to add a UNIQUE constraint on the email column of the student table

SQL> ALTER TABLE student ADD UNIQUE (email);

Output:?

CREATE STUDENT_1 TABLE:

SQL> CREATE TABLE student_1 (
 sid INT,
 name VARCHAR2(30),
 score INT,
 email VARCHAR2(50),
 deptid INT
);

Outout:?

Query: Write a query to add a primary key to the student_1 table

SQL> ALTER TABLE student_1 ADD PRIMARY KEY (sid);

Output:?

Method 2: ADD constraint WITH constraint name

Syntax:

ALTER TABLE table_name ADD CONSTRAINT constraint_name constraint_type
(column_name);

Query: Write a query to add a UNIQUE constraint on the email column of the student_1 table

SQL> ALTER TABLE student_1 ADD CONSTRAINT unique_email UNIQUE (email);

Output:?

ALTER TABLE (DROP CONSTRAINT)

Query: Write a query to drop constraints from the student table.

Method 1: DROP PRIMARY KEY

Syntax:

ALTER TABLE table_name DROP PRIMARY KEY;

SQL> ALTER TABLE student_1 DROP PRIMARY KEY;

Output:?

Method 2: DROP constraint using constraint name

Syntax:

ALTER TABLE table_name DROP CONSTRAINT constraint_name;

SQL> ALTER TABLE student_1 DROP CONSTRAINT unique_email;

Output:?

FOREIGN KEY

Inline Foreign Key

Query: Write a query to create dept_2 table and add foreign key in student_2 table.

Syntax:

```
column_name datatype REFERENCES parent_table(parent_column);
```

```
SQL> CREATE TABLE dept_2 (  
    deptid INT PRIMARY KEY,  
    dname VARCHAR2(20)  
);  
SQL> INSERT INTO dept_2 VALUES (101, 'CSE');  
SQL> INSERT INTO dept_2 VALUES (102, 'ECE');  
SQL> CREATE TABLE student_2 (  
    sid INT PRIMARY KEY,  
    name VARCHAR2(30) NOT NULL,  
    marks INT,  
    email VARCHAR2(30) UNIQUE,  
    deptid INT REFERENCES dept(deptid)  
);  
SQL> INSERT INTO student_2 VALUES (1, 'Ravi', 85, 'ravi@gmail.com', 101);  
SQL> INSERT INTO student_2 VALUES (2, 'Sita', 92, 'sita@gmail.com', 102);
```

Output:?

Check referential integrity:

```
SQL> INSERT INTO student VALUES (3, 'Anita', 88, 'anita@gmail.com', 103);
```

Output:?

```
ORA-02291: integrity constraint (SYSTEM.STUDENT_DEPTID_FK) violated - parent  
key not found
```

ALTER TABLE (RENAME TABLE)

Syntax:

```
ALTER TABLE old_table_name RENAME TO new_table_name;
```

Query: Write a query to rename the student table to student_details.

```
SQL> ALTER TABLE student RENAME TO student_details;
```

SQL>ALTER TABLE dept RENAME TO department;

Output:?

DROP TABLES

Syntax:

DROP TABLE table_name;

Query: Write a query to drop the student and dept tables.

SQL> DROP TABLE student;

SQL> DROP TABLE dept;

Output:?
